Museum Guard Shift – Draft Report

Group Members: Dmitri Lvovich, Raphaël Michnik

Report

Intended solution and explanation for why it works

The intended solution is:

1. preprocess the tree so that distances between important rooms can be queried quickly,
2. run dynamic programming over subsets of alarmed rooms.

The important rooms are:

- the starting room S,
- the K alarmed rooms.

Because the graph is a tree, the shortest path distance between any two rooms is just the path length in the tree. We can root the tree anywhere, preprocess depths and root-distances, and use lowest common ancestor (LCA) to compute

$$\text{dist}(u, v) = \text{distRoot}[u] + \text{distRoot}[v] - 2 \cdot \text{distRoot}[\text{lca}(u, v)].$$

This lets us build a complete weighted graph on the K alarmed rooms plus the start room, where the edge weights are travel times in the original tree.

Then we define a dynamic programming state

$$dp[\text{mask}][i]$$

where:

- `mask` indicates which alarmed rooms have already been handled,
- i is the last handled alarmed room,
- $dp[\text{mask}][i]$ is the earliest possible time at which the guard can finish handling all alarms in `mask` and end at room i.

Initialization. For each alarmed room i, the guard can go there directly from S. So the initial completion time is

$$\text{dist}(S, i) + a_i.$$

If that time is at most d_i, then

$$dp[1 \ll i][i] = \text{dist}(S, i) + a_i.$$

Otherwise that state is invalid.

Transition. Suppose we are at state $dp[\text{mask}][i]$, and want to handle a new alarm j not in `mask`. The completion time would be

$$dp[\text{mask}][i] + \text{dist}(i, j) + a_j.$$

This transition is valid only if the above quantity is at most d_j. If valid, update

$$dp[\text{mask} \cup \{j\}][j] = \min\Big(dp[\text{mask} \cup \{j\}][j],\ dp[\text{mask}][i] + \text{dist}(i, j) + a_j\Big).$$

Answer. If $K = 0$, the answer is 0. Otherwise, the answer is

$$\min_i dp[(1 \ll K) - 1][i].$$

If all such states are unreachable, print `IMPOSSIBLE`.

Why it works

This dynamic programming is correct because every feasible solution has some last handled alarm. If we know the set of already handled alarms and the last one visited, then the only information needed for future decisions is the current completion time and current room. Since $dp[\text{mask}][i]$ stores the earliest such completion time, it dominates any slower way to reach the same state.

This gives an optimal substructure: every optimal full schedule consists of an optimal schedule for some smaller subset, followed by one final valid transition.

Thus the recurrence explores exactly all feasible orders of alarm handling, while pruning dominated states by keeping only the earliest completion time for each (mask, i).

Complexity

Let K be the number of alarmed rooms.

- Tree preprocessing with DFS/BFS + LCA: $O(N \log N)$
- Pairwise distances between important rooms: $O(K^2 \log N)$, or $O(K^2)$ after preprocessing with $O(1)$ LCA queries
- Dynamic programming: $O(2^K K^2)$

This is feasible for $K \leq 18$.

Any intended problem pitfalls or incorrect approaches

A few natural wrong ideas:

1. **Greedy by nearest room.** Choosing the closest unhandled alarm is not always optimal. A slightly farther room may have a much tighter deadline, so going to the nearest first can make the instance fail even when a valid schedule exists.
2. **Greedy by earliest deadline.** Likewise, always going to the smallest deadline first can fail because the travel time between rooms matters. The order must balance both current position and remaining deadlines.

3. **Treating it as ordinary shortest path.** This is not a plain shortest path problem because the guard must choose an order over multiple required targets, and deadlines constrain which permutations are feasible.
4. **Running all-pairs shortest paths on the entire tree.** That would be unnecessary and too expensive for large N. The graph is a tree, so distances should be obtained using tree preprocessing, not Floyd–Warshall or repeated Dijkstra.
5. **Checking deadline at arrival instead of completion.** The statement says the alarm is handled only after spending a_i time opening the panel. So the deadline applies to finishing the action, not merely reaching the room.

Any particular design choices

1. **Tree structure with small K.** The large tree keeps the problem visually interesting and prevents trivial dense-graph solutions, while the small number of alarmed rooms keeps the intended dynamic programming manageable and contest-appropriate.
2. **Weighted edges.** Weighted corridors make the problem less like a simple BFS ordering problem and force contestants to reason about true travel time.
3. **Start room may be alarmed.** This is a good edge case and makes the statement slightly richer without adding complexity.
4. **Finish anywhere.** The guard does not need to return to the start. This makes the optimization cleaner and avoids adding a redundant final leg.
5. **Distinct alarmed rooms.** This removes ambiguity in the input and makes the dynamic programming state definition simpler.
6. **Readability.** The statement is intentionally compact: tree, start room, alarm rooms with open times and deadlines, minimum finish time or impossible. The algorithmic core then becomes easy to explain.

Descriptions of planned test cases needed to verify correctness and efficiency

We would include both hand-crafted and generated tests.

Hand-crafted correctness tests.

- No alarms: $K = 0$, answer should be 0.
- Single alarm, feasible: direct travel plus handling fits deadline.
- Single alarm, impossible: deadline missed even if visited first.
- Start room is alarmed: handling can begin immediately with travel time 0.
- Exact deadline equality: finish time exactly equals deadline, verifying $\leq$ instead of $<$.
- Chain-shaped tree: tests long unique routes.
- Star-shaped tree: all alarms branch from a center, good for exposing greedy mistakes.
- Balanced binary tree: structured but nontrivial distances.

- Must revisit shared ancestors: alarms in different branches, ensuring solutions correctly use tree distances through LCA structure.
- Impossible only because of ordering: every alarm individually reachable, but no global order works.
- Multiple valid orders but different total times: verifies optimization, not just feasibility.
- Greedy trap: nearest-first fails, optimal order succeeds.

Performance and stress tests.

- Large random tree, small K: large N, random weighted edges, moderate deadlines.
- Maximum K: $K = 18$, distances arranged so many dynamic programming states are reachable.
- Large irrelevant tree: very large N but alarms concentrated in a small region, checking preprocessing overhead and robustness.
- Large path lengths: weights large enough to require 64-bit arithmetic.

Plans on generating said test cases

We would generate tests in several ways:

1. **Manual examples.** Small examples written by hand for sample cases, deadline edge cases, greedy-failure constructions, and the start-room-alarmed case.
2. **Structured generators.** We would write generators for chains, stars, balanced binary trees, broom trees (a long path with many leaves at one end), and random trees via Prüfer sequences.
3. **Deadline generation.** For each generated instance, choose an intended visiting order and compute its exact finish times, then create several variants: deadlines comfortably above finish times, deadlines exactly equal to finish times, deadlines lower by 1 on one room to create near-failure, and mixed loose and tight deadlines.
4. **Brute-force validator for small cases.** For small N and small K, for example $K \leq 8$, generate random instances and compare the intended solution against a brute-force checker that tries all $K!$ alarm orders. This is especially useful for validating the dynamic programming implementation and catching off-by-one deadline mistakes.

An argument for why those test cases are sufficient

These tests cover both correctness and efficiency.

For correctness, they cover:

- trivial instances,
- edge cases,
- exact-boundary behavior,
- multiple tree shapes,
- cases requiring backtracking across the tree,

- cases where greedy reasoning fails,
- both feasible and impossible instances.

For efficiency, they cover:

- very large trees,
- maximum number of alarms,
- large edge weights,
- random and adversarial structures.

The brute-force cross-check on small random cases provides strong evidence that the recurrence and transitions are correct, while the larger structured cases verify that the implementation scales to the intended constraints.

Taken together, these tests exercise every key aspect of the problem:

- tree distance handling,
- deadline interpretation,
- subset dynamic programming state transitions,
- 64-bit arithmetic,
- asymptotic performance.